ADAPT-UAP: Spectrally Weighted Truncated Ratio Maximisation for Data-Free Universal Adversarial Perturbations

Sushant Shrestha
Independent Researcher
Kathmandu, Nepal
sushant.901.sus@gmail.com

Abstract

A data-free Universal Adversarial Perturbation (UAP) is a single image-agnostic perturbation, bounded in ℓ_∞ norm, computed from a trained network's weights alone without access to any training or test image. The setting is practically important because it models the realistic threat in which an attacker obtains a deployed checkpoint but not the data it was trained on, and it is scientifically interesting because whatever such an attack exploits must be a property of the learned weights rather than of the data distribution. We present ADAPT-UAP, which combines four ideas that the existing data-free literature has developed separately. The objective is a per-layer ratio between a truncated aggregate of positive activations and the magnitude of the activations that the rectifier discards, and it is weighted across layers by a spectral summary of each layer's weight matrix computed once from the checkpoint. Crafting proceeds against synthetic inputs carrying structure at three spatial scales simultaneously, under a perturbation budget that is raised in stages over the run with the perturbation rescaled rather than clipped at each transition. We give the full specification, including every choice that the description alone does not determine, together with a pre-registered evaluation protocol over six methods, eight architectures and five seeds on the complete ImageNet validation set, and a frozen evaluation harness whose preprocessing is fingerprinted so that a reported fooling rate can be traced to the exact pipeline that produced it. We reimplement five prior methods inside that harness to serve as baselines and publish the implementation notes, including a measurement of an activation-capture hazard that silently removes the denominator of a ratio objective on architectures where a convolution feeds an in-place rectifier, along with a one-line test for it. The implementation is released in full.

Status. This version presents the method, the baseline implementations and the pre-registered protocol. The benchmark that protocol specifies is in progress; its tables are given here in the form they will be reported in, with every cell marked pending, and they will be populated in a later version of this paper. No empirical claim about ADAPT-UAP's accuracy is made anywhere in this version.

***Index Terms*—universal adversarial perturbation, data-free attack, adversarial robustness, spectral analysis, reproducibility**

Version 1, October 2026. This version is the method, the baseline implementations and the pre-registered protocol; Section VI holds the result tables in the form they will be reported in, with every cell marked *pending* until the benchmark of Section V has been executed. A later version will populate them. Code, decision log and evaluation harness are released with this version.

I. Introduction

Universal Adversarial Perturbations [1] are single perturbations that, added to most images drawn from a distribution, change a classifier's prediction. The original construction needed a sample of that distribution. A subsequent line of work removed that requirement entirely, and it is that data-free line this paper contributes to.

Its attraction is twofold. Operationally, an attacker who has obtained a deployed checkpoint, whether from a model zoo, a shipped mobile application or an inference endpoint, very often has no access at all to the proprietary data that produced it, so a data-free attack is the one that describes the realistic threat. Scientifically, an attack that works from weights alone cannot be exploiting a quirk of the data distribution, so whatever it exploits is a property of the learned function, and understanding it tells us something about the representation rather than about the dataset.

Progress in the setting has been rapid. Fast Feature Fool [2] established that maximising intermediate activations under a constant synthetic input alone yields a perturbation with a fooling rate around 47% on VGG-16. GD-UAP [3] generalised the objective and the task coverage. TRM-UAP [4] observed that the negative, discarded half of each rectified activation carries information the objective had been ignoring, and turned the objective into a ratio. PSP-UAP [5] refined the synthetic input. IntriUAP [7] brought the spectral structure of the layers themselves into the picture, aligning the propagated perturbation with the direction of greatest amplification.

These contributions are complementary rather than competing. Each identifies a different lever: *what* to maximise, *where* in the network to maximise it, *what to show* the network while doing so, and *how* to manage the perturbation budget during the search. ADAPT-UAP is the result of pulling all four levers at once, and this paper's contributions are:

1) **A method, specified completely.** Section III gives ADAPT-UAP's four components with their exact functional forms, and Section III-F additionally records every choice that the specification alone does not fix, with the reason for the value taken.

2) **A design in which the contested choices are configuration, not code.** Three of ADAPT-UAP's design questions do not have settled answers. Rather than pick one and report it as though it were determined, each candidate is implemented and selected by a configuration key, so the ablation that settles the question costs a configuration file.
3) **A pre-registered evaluation protocol** (Section V): six methods, eight architectures, five seeds, the full 50,000 image ImageNet validation set, with the comparisons, statistical tests and multiple-comparison correction all fixed in advance of the run.
4) **A frozen, fingerprinted evaluation harness** (Section V-E), which makes a fooling rate reported here traceable to the exact preprocessing pipeline that produced it.
5) **Five baselines reimplemented in that harness, with their implementation notes published** (Section IV). Reimplementing a method from its released code raises questions the published description does not answer, and a comparison is only as fair as the answers. We give ours, including a measurement of an activation-capture hazard (Section IV-D) that silently removes one half of a ratio objective on some architectures and that any implementation built the same way will meet.

Section VI is deliberately unpopulated in this version. The benchmark requires roughly 206 GPU-hours and has not yet been run, and a method paper whose tables are filled with numbers from a smaller or different experiment than the one it describes is exactly the failure mode the rest of this paper is built to avoid.

II. Background

A. Setting

Let f be a trained classifier and $\mathcal{X}$ the distribution of images it sees at deployment. A universal perturbation is a single v with $\|v\|_\infty \le \varepsilon$ such that $f(x+v) \neq f(x)$ for a large fraction of $x \sim \mathcal{X}$. The quantity reported is the *fooling rate*

$$\mathrm{FR}(v) \;=\; \mathbb{E}_{x\sim\mathcal{X}}\big[\,\mathbf{1}\{f(x+v) \neq f(x)\}\,\big], \tag{1}$$

measured against the model's own clean prediction rather than against the ground-truth label, following [1]. The two definitions differ by roughly the model's error rate and are not interchangeable; both are recorded here and the clean-prediction definition is the headline.

The setting is *data-free* when the procedure computing v never evaluates f on a real image. Throughout, $\varepsilon = 10/255$ in $[0,1]$ pixel space, the field's standard budget, and v is added before normalisation so that the budget means ten quantisation levels of an 8-bit image rather than a channel-asymmetric multiple of it.

B. What the existing methods contribute

Each of the four levers ADAPT-UAP pulls was identified by earlier work, and it is worth stating precisely what each contributes, because ADAPT-UAP's design is a claim about how they combine.

Activation maximisation as the objective. Fast Feature Fool [2] showed that a perturbation which drives intermediate convolutional activations to large magnitudes, under nothing but a constant input, transfers to real images. GD-UAP [3] extended this across tasks.

The discarded half of the rectifier. TRM-UAP [4] observed that a ReLU network's response to a perturbation has two parts: the activations that survive rectification and those driven below zero and discarded. Maximising the first while suppressing the second is strictly more informative than maximising the first alone, and truncating the aggregate so that it is dominated by strongly responding units focuses the objective further.

Structured synthetic inputs. A data-free attack must show the network something. TRM-UAP adds Gaussian or block-shuffled fields to the perturbation before each forward pass, and Cosine-UAP's ablation [6] finds that locally flat, piecewise-constant synthetic images outperform uniform noise, Gaussian noise and flat inputs. The evident interpretation is that flat regions separated by sharp edges are what a network's early filters are tuned for.

The spectrum of the layers. IntriUAP [7] takes the singular value decomposition of each linear operator and aligns the propagated perturbation with the right singular vector of the largest singular value, that being the input direction which maximises the ℓ_2 norm of the change in that layer's output. This introduces the idea that the layers are not equally worth attacking and that the weights themselves say which are.

ADAPT-UAP takes the ratio objective from the second, the structured synthetic input from the third, the idea that layers should be weighted by a spectral property from the fourth, and adds a budget schedule of its own. The remainder of the paper is about how they fit together and what has to be decided to make them fit.

III. ADAPT-UAP

Write $a_\ell(x) \in \mathbb{R}^{n_\ell}$ for the output of the ℓ-th convolutional layer on input x, flattened, and $a^+ = \max(a,0)$, $a^- = \min(a,0)$ for its rectified and discarded parts. Let x_0 denote the crafting input, which contains no real image content, and L the number of convolutional layers.

A. Component A: spectral layer weighting

Each layer receives a weight computed once, from the checkpoint alone, before any optimisation begins. Unfold layer ℓ's weight tensor into the matrix $W_\ell \in \mathbb{R}^{C_{\mathrm{out}} \times C_{\mathrm{in}} k_h k_w}$ whose action on an unfolded input patch is the layer's linear map, and let $\sigma_{\max}(\ell) \ge \cdots \ge \sigma_{\min}(\ell)$ be its singular values. With q_ℓ a scalar summary of that spectrum,

$$w_\ell \;=\; \frac{\log(1+q_\ell)}{\sum_{k=1}^{L}\log(1+q_k)}, \qquad \sum_\ell w_\ell = 1. \tag{2}$$

The logarithm is not cosmetic. On VGG-16 the condition number rises from roughly 12 in the first block to above 1,000

in the deepest, and an unlogged weighting would hand a single layer essentially the entire objective.

Three choices of q_ℓ are implemented and selectable: $q_\ell = \kappa_\ell = \sigma_{\max}(\ell)/\sigma_{\min}(\ell)$, the condition number; $q_\ell = \sigma_{\max}(\ell)$, the quantity IntriUAP's argument directly concerns; and $q_\ell = 1$ for all ℓ, which makes (2) uniform and serves as the control. We report all three. The distinction is real rather than pedantic: a layer that amplifies strongly in every direction has a large $\sigma_{\max}$ and a *low* κ, while a layer with one nearly null direction has an enormous κ and unremarkable amplification, so the two summaries can order the layers differently. Which ordering produces better perturbations is an empirical question, and Section V-D asks it.

One numerical point matters more than its obscurity suggests. Unfolded convolutional kernels are routinely rank deficient, so the smallest singular value is frequently an exact zero and κ is then infinite, which after the normalisation in (2) makes *every* weight undefined. We therefore take $\sigma_{\min}(\ell)$ to be the smallest singular value above the standard numerical rank tolerance $\max(m, n) \cdot \epsilon_{\text{dtype}} \cdot \sigma_{\max}$, so that κ is the condition number of the layer's effective rank, and we record that rank alongside it.

B. Component B: truncated ratio maximisation

For each layer, define a truncated aggregate of the rectified activations and a magnitude for the discarded ones. Let $T_k(a^+)$ be the largest k fraction of the entries of a^+. The positive term is a temperature-softened power mean over that retained set,

$$P_\ell \;=\; \Big(\tfrac{1}{|T_k|}\textstyle\sum_{i\in T_k} a_i^{1/\tau}\Big)^{\tau}, \tag{3}$$

and the negative term is the root mean square of the discarded part,

$$N_\ell \;=\; \big(\tfrac{1}{n_\ell}\|a_\ell^-\|_2^2\big)^{1/2}. \tag{4}$$

The objective, minimised in v, is the negative of a spectrally weighted sum of per-layer log ratios:

$$\mathcal{L}(v) \;=\; -\sum_{\ell=1}^{L} w_\ell\big[\log P_\ell(x_0+v) - \log N_\ell(x_0+v)\big]. \tag{5}$$

Two aspects of this form are deliberate and are easy to get wrong.

The ratio is per layer, and the weighting is applied to it. The alternative, pooling numerator and denominator across layers and taking one ratio, would reduce Component A's weights to what is close to a global rescaling of the loss, which an adaptive optimiser absorbs. Component A would then cost a singular value decomposition of every weight tensor, appear in the configuration, appear in this paper, and change nothing measurable. In the form of (5) the weights determine which layer's ratio the gradient serves.

Both terms are size-independent. (3) is a mean and (4) a root mean square, so neither scales with a layer's element count. Mixing a sum and a norm, as one naturally might, introduces into (5) an additive per-layer constant that varies with feature map resolution, which would weight an early high-resolution layer differently from a deep low-resolution one for a reason unrelated to either method's intent.

The temperature τ in (3) interpolates between a plain mean at $\tau = 1$ and a maximum as $\tau \to 0$; at the value used, $\tau = 0.1$, (3) is the tenth-power mean, dominated by the strongly responding units while still passing gradient to every retained one. It is computed in log space as $\tau\,(\operatorname{logsumexp}(\log a/\tau) - \log|T_k|)$, since a^{10} overflows to no purpose. Section III-F records why the superficially similar softmax-weighted alternative is not usable here.

C. Component C: multi-scale pseudo-semantic priors

The crafting input is

$$x_0 \;=\; \operatorname{clip}_{[0,1]}\big(\mu + s\cdot(\pi_t - \tfrac{1}{2})\big), \tag{6}$$

where μ is the constant per-channel dataset mean image, s a strength, and π_t a synthetic field redrawn at every iteration t. Centring on μ rather than on black matters: the reference implementations operate in mean-subtracted space, so the input at which they evaluate their objective is the mean image, and using black instead would place the network roughly two normalised units away from that point and change every activation the objective is built from. Shifting π_t by $-\frac{1}{2}$ keeps x_0 centred on μ rather than pressed against the upper clip.

The field itself carries structure at three spatial scales at once. At each scale $s_j \in \{224, 64, 32\}$ we draw a grid of independent uniform colours, expand it so that each block is exactly flat, and, for s_j smaller than the input, tile the resulting $s_j \times s_j$ field by repetition to full size; the three are then averaged. Tiling is by repetition and never by interpolation, because an interpolating upsample smooths away precisely the flat edges the field exists to provide. A tile smaller than a deep layer's receptive field presents that layer with the same pseudo-object repeated across its whole input, which is the behaviour the component is for, while the full-scale component retains the global statistics a single small tile cannot express.

D. Component D: progressive budget scheduling

The budget is raised in stages over the run, $\varepsilon : 2/255 \to 4/255 \to 8/255 \to 10/255$, rather than held at its final value throughout. At a transition the perturbation is *rescaled*,

$$v \;\leftarrow\; v\cdot\varepsilon_{\text{new}}/\varepsilon_{\text{old}}, \tag{7}$$

rather than simply permitted to grow. The distinction is not cosmetic. Under an ℓ_∞ constraint most coordinates sit at the bound, and it is the *direction* of v, not its unsaturated coordinates, that the preceding stage's optimisation bought; (7) carries that direction into the larger budget intact. The truncation fraction k of (3) decays along the same schedule, from 0.30 to 0.05, so that the objective broadens its attention early and concentrates it as the budget and the perturbation grow.

Algorithm 1 ADAPT-UAP

Require: frozen f; budget ε; schedule S; iterations T
Ensure: v with $\|v\|_\infty \le \varepsilon$

$w_{1..L} \leftarrow$ spectral weights of f ▷ Eq. (2), once
$\varepsilon_0 \leftarrow S(1)$; $v \sim \mathcal{U}(-\varepsilon_0, \varepsilon_0)$
for $t = 1 \dots T$ **do**
  $e \leftarrow$ stage index of t
  if $S(e) \neq \varepsilon_{\text{cur}}$ **then**
    $v \leftarrow v \cdot S(e)/\varepsilon_{\text{cur}}$; $\varepsilon_{\text{cur}} \leftarrow S(e)$ ▷ Eq. (7)
  end if
  $k \leftarrow k(e)$ ▷ decayed truncation fraction
  $x_0 \leftarrow$ multi-scale prior ▷ Eq. (6)
  $\mathcal{L} \leftarrow$ weighted truncated ratio ▷ Eq. (5)
  $v \leftarrow \mathrm{Adam}(v, \nabla_v \mathcal{L})$
  $v \leftarrow \mathrm{clip}(v, \pm\varepsilon_{\text{cur}})$
  if v saturated and saturation has plateaued **then**
    $v \leftarrow v/2$ ▷ Sec. III-F
  end if
end for
return v

E. The complete procedure

Algorithm 1 evaluates f only on $x_0 + v$, and x_0 is synthesised. No dataset is read, and no parameter of f is updated. Both properties are enforced structurally in the implementation rather than promised: the crafting interface receives a model and a configuration and is never handed data, and the released test suite additionally fails if a data loader is constructed anywhere inside a crafting call.

F. Choices the specification does not fix

Some decisions necessary to run the method are not determined by its description. We record them here rather than leaving them implicit, since they are exactly the decisions that make an independent reimplementation produce a different number.

A saturation escape is required. Adam at learning rate 0.1 acting on a variable clipped to $2/255$ pins essentially every coordinate at the bound within a few steps, after which the gradient carries little usable information. This is why the reference methods halve the perturbation periodically [2] or on a saturation plateau [3]. Component D does not substitute for this: (7) maps a coordinate pinned at ε_{old} exactly onto ε_{new}, so the saturation fraction is invariant across a stage transition by construction. ADAPT-UAP therefore adopts the saturation-plateau rule, and we report its removal as an ablation.

τ is a power-mean exponent, not a softmax temperature. The natural alternative reading of a temperature, a softmax-weighted mean $\sum_i \mathrm{softmax}(a/\tau)_i\, a_i$, does not work at this scale. Activation magnitudes are of order 1 to 10, so $a/0.1$ spans order 100, the softmax collapses onto a single element, the aggregate degenerates to the per-layer maximum, the truncation fraction k becomes inert because the largest value belongs to every retained set, and the gradient reaches one activation per layer per step. The power mean of (3) has none of these properties because its concentration is polynomial rather than exponential.

Adam rather than a sign-gradient step. All five comparison methods optimise the perturbation with Adam at learning rate 0.1. ADAPT-UAP does the same, so that a difference in fooling rate is attributable to Components A to D rather than to the optimiser. The sign-gradient alternative is implemented and reported as an ablation.

Activations are captured at convolution outputs and rectified explicitly. The reference implementations were written against frameworks whose layer dictionaries hold post-rectification activations. Capturing convolution outputs and applying the rectifier reproduces that quantity exactly where a convolution is followed directly by a rectifier, and approximates it where batch normalisation intervenes. Applied uniformly to all six methods, this keeps the comparison measuring one thing.

IV. Baselines and how they were implemented

A benchmark is only as fair as its baselines, and reimplementing a method from released code raises questions the paper accompanying it does not answer. This section records ours, so that a reader can judge the comparison rather than take it on trust, and so that anyone reimplementing these methods does not have to rediscover the same answers.

Nothing here is offered as a correction to the original work. These are the decisions a reimplementer faces, and the value of writing them down is that the next person does not face them alone.

A. One evaluation pipeline for all six methods

Two of the reference implementations optimise their perturbation against un-normalised network input while measuring the fooling rate on normalised input. Reproducing that faithfully would mean each method crafting in a different space from the one it is scored in, and two of the six differing from the other four. Because the purpose here is a comparison, every method instead crafts and is evaluated through the same frozen pipeline of Section V-E. Each method's objective, priors and curricula are otherwise reproduced as released.

B. Data-free means the interface holds no data

Two reference training loops measure the fooling rate on real validation images at intervals during crafting, and use it to stop early, to choose a rescaling factor, and in one case to select which iterate to return. The images are unlabelled, since the fooling rate is measured against the model's own clean prediction, but they are real.

This benchmark's attack interface receives a model and a configuration and never receives data, so that the data-free property is structural rather than promised; the released test suite additionally fails if a data loader is constructed inside a crafting call. Those two methods therefore run here for a fixed iteration count with no measurement-driven stopping. The consequence is worth stating plainly: **their numbers here are not directly comparable with their published ones**, and

TABLE I

ACTIVATION-CAPTURE HAZARD. WHERE A CONVOLUTION FEEDS AN IN-PLACE RECTIFIER, THE NEGATIVE TERM COLLAPSES TO THE CONSTANT $n_{\text{keep}} \log \epsilon$ AND CARRIES NO GRADIENT. BATCH NORMALISATION BETWEEN THE TWO ALLOCATES A NEW TENSOR, WHICH PREVENTS IT.

Architecture	convs	no neg.	as captured	copied first
VGG-16	13	**13**	**−207.23**	+24.44
VGG-19	16	**16**	**−248.68**	+27.53
AlexNet	5	**5**	**−82.89**	+5.62
ResNet-152	155	0	+1191.51	+1191.51
GoogLeNet	59	0	−606.94	−606.94
DenseNet-121	120	0	+251.00	+251.00

The three collapsed values equal $n_{\text{keep}} \log(10^{-9})$ exactly, for $n_{\text{keep}} = 10, 12, 4$.

a shortfall should be read as the cost of the stricter setting rather than as a failure to reproduce.

C. Hyperparameters follow the published per-architecture values

One reference records its best settings per architecture in a comment block rather than in its argument defaults, and the two differ. This benchmark uses the per-architecture values, so each baseline is compared under the settings its authors report. Three of the eight architectures here were not tuned by that work and fall back to its defaults; those cells are marked as such in the results rather than presented as though they were tuned.

D. An activation-capture hazard, and how to detect it

Every method in this benchmark reads intermediate activations through forward hooks. A hook receives a *reference* to a layer's output, not a copy, and the standard PyTorch vision models construct their rectifiers with `inplace=True`. Wherever a convolution feeds a rectifier directly, the rectifier therefore overwrites the convolution's output buffer before the forward pass returns, and any objective reading that tensor afterwards sees a value from which every negative entry has already been removed.

For an objective built on the positive part this costs nothing, since rectifying twice is rectifying once. For an objective built on the *negative* part, which is what a truncated ratio uses as its denominator, it removes the term entirely. Table I measures the effect using the negative term of the truncated-ratio family, $\sum_l \log(\|\min(a_l, 0)\|_2/2+\epsilon)$, at that family's default truncation rate.

Three properties of this make it worth a table rather than a footnote. It is **architecture-dependent**, so a run that looks correct on one network is not evidence about another. It is **silent**: the run completes, the loss decreases, and the fooling rate is plausible, because a weighting on a constant term still produces a number. And it is **invisible to the positive part**, which is why an implementation can pass every check built around the numerator.

The fix is one line, copying the tensor inside the hook, and it must be a copy that preserves the autograd graph rather than a detached one, since the objective differentiates through these activations. The detection is equally cheap and is what we recommend: compute the negative term once and test whether it equals $n_{\text{keep}} \log \epsilon$. We release the script that produces Table I; it needs no GPU, no dataset and no trained weights, because whether a rectifier operates in place is a property of the module and not of the values flowing through it.

ADAPT-UAP's own denominator is subject to the same hazard, and Table I is as much a record of a mistake we had to find in our own implementation as it is guidance for anyone else's. We found it only because an ablation over the ratio's weighting refused to change any result.

V. EVALUATION PROTOCOL

This section is pre-registered: it fixes the comparisons, the statistics and the correction before the run, so that the analysis cannot be selected after seeing the numbers.

A. Scope

Six methods (FFF, GD-UAP, TRM-UAP, PSP-UAP, Cosine-UAP, ADAPT-UAP) by eight architectures (AlexNet, VGG-16, VGG-19, GoogLeNet, ResNet-152, DenseNet-121, Swin V2-S, DeiT-III-S) by five seeds, giving 240 cells and an estimated 206 GPU-hours. A random perturbation of equal budget is evaluated in every cell as a floor. Evaluation uses the complete 50,000 image ImageNet validation set; a subsample of a few thousand images carries a sampling error of a magnitude comparable to the differences being tested, which is not adequate for the comparisons below.

B. Metrics

Primary: fooling rate against the clean prediction, (1). Secondary and reported alongside: fooling rate against ground truth; the full 6×8 cross-model transfer matrix with each perturbation crafted on one architecture and evaluated on all eight; the transfer gap; a budget sweep over $\varepsilon \in \{4, 6, 8, 10, 12\}/255$; and convergence, measured as the number of iterations to reach a fixed fraction of the run's final fooling rate.

C. Statistics

For each architecture, ADAPT-UAP is compared against each baseline by a paired comparison over the five seeds, using both a paired bootstrap over images and a Wilcoxon signed-rank test over seeds, with Holm-Bonferroni correction across the family of comparisons. Effect sizes are reported with confidence intervals. Five seeds support a signed-rank test at its minimum useful size and no more; differences smaller than roughly two percentage points should be read as unresolved by this design rather than as absent.

D. Ablations

Each component is removed or varied independently, at three seeds, on VGG-16, ResNet-152 and Swin V2-S:

- **A**: κ-weighted against $\sigma_{\max}$-weighted against uniform. This distinguishes "spectral weighting helps" from "this

TABLE II
FOOLING RATE (%) AGAINST THE CLEAN PREDICTION, $\varepsilon = 10/255$, FULL IMAGENET VALIDATION SET, MEAN OVER FIVE SEEDS.

Method	VGG-16	ResNet-152	Swin V2-S
Random noise	*pending*	*pending*	*pending*
FFF	*pending*	*pending*	*pending*
GD-UAP	*pending*	*pending*	*pending*
TRM-UAP	*pending*	*pending*	*pending*
PSP-UAP	*pending*	*pending*	*pending*
Cosine-UAP	*pending*	*pending*	*pending*
ADAPT-UAP	*pending*	*pending*	*pending*

Three of eight architectures shown; the full 6×8 table with standard deviations appears in the supplement.

TABLE III
COMPONENT ABLATION ON VGG-16, THREE SEEDS. CHANGE IN FOOLING RATE (PERCENTAGE POINTS) FROM THE FULL METHOD.

Variant	Δ FR
Full method	—
A: $\sigma_{\max}$ instead of κ	*pending*
A: uniform layer weights	*pending*
B: layer-prefix truncation	*pending*
B: positive term only, no ratio	*pending*
C: full scale only	*pending*
C: no prior (constant input)	*pending*
D: fixed budget	*pending*
D: clip instead of rescale at transition	*pending*
D: no saturation escape	*pending*
Sign-gradient instead of Adam	*pending*

spectral summary is the right one", which the more usual weighted-against-uniform comparison cannot.

- **B**: value-level truncation against layer-prefix truncation; τ swept; the ratio replaced by the positive term alone.
- **C**: three scales against full scale alone against no prior.
- **D**: staged budget against fixed budget; rescale at transition against clip at transition; saturation escape present against absent.

E. The evaluation harness

Every method in the comparison is evaluated through one frozen preprocessing pipeline: resize the shorter side to 256, centre-crop to 224, scale to $[0, 1]$, add v, clip to $[0, 1]$, normalise. The order is load-bearing, since adding v after normalisation would scale the effective budget by the reciprocal of the per-channel standard deviation and make it channel-asymmetric. The pipeline is fingerprinted twice, once over its declared constants and once over its measured behaviour on a fixed probe input, and both fingerprints travel inside every result record. A change to the pipeline therefore invalidates the records it would otherwise silently contaminate.

Each cell records its configuration hash, both preprocessing fingerprints, the git commit, the resolved dependency versions, the device, and the crafting trace. Runs are resumable through a lock-protected manifest with atomic writes and leases, so that an interrupted 206 GPU-hour job resumes rather than restarts, and a cell that fails is quarantined rather than silently retried.

VI. PRE-REGISTERED RESULT TABLES

This section contains no measurements. The benchmark of Section V has not been executed: it requires pretrained checkpoints, the ImageNet validation set and roughly 206 GPU-hours, none of which were available during development. Every cell below is marked *pending*. The tables are published in this state deliberately, so that the protocol is fixed in public before the numbers exist. No claim about ADAPT-UAP's accuracy is made anywhere in this paper.

What *can* be reported at this stage concerns the implementation rather than the method. On a small convolutional network with synthetic inputs, run on CPU, ADAPT-UAP respects its budget exactly, is deterministic in its seed, trains no parameter of the target, advances its budget schedule through all four stages, decays its truncation fraction as specified, reduces its own objective when that objective is measured on synthetic inputs held out from crafting, and exceeds an equal-budget random perturbation. These are correctness properties of the code. They are not evidence about the method, and we draw no inference from them: the test network has random weights and no biases, and its fooling rates are an artefact of the fixture.

VII. LIMITATIONS

No empirical result. The central limitation is the one stated above. Until Section VI is populated, this paper is a specification and a protocol, and its claims are confined to those.

Three design questions are open. Whether the condition number or the largest singular value is the right spectral summary, whether truncation should act on values or on layers, and whether Adam or a sign-gradient step should be used, are not settled here. Each is implemented as a selectable mode with a documented default so that the ablations of Section V-D settle them, but until those ablations run, the defaults are recommendations.

The spectral motivation does not extend to the transformer targets by argument. The reasoning that makes a layer's spectrum predictive of its contribution to vulnerability relies on the non-linear operators between layers being non-expansive, which holds for the rectified convolutional architectures but not for softmax attention, GELU or layer normalisation. Two of the eight architectures therefore sit outside it. The singular value decomposition of an attention projection is perfectly well defined, and Component A may well work there; if it does, that is an empirical finding about a heuristic exceeding the scope of its own motivation, and we will report it as such rather than implying the motivation carried over.

Five seeds. Adequate for the pre-registered tests and no more, as Section V states.

A. Status of this version, and what the next one adds

Publishing a specification and its protocol before the measurement is a deliberate choice rather than an accident of

timing. A pre-registration is only worth the name if it predates the data, and the alternative, holding everything back until the tables are full, makes the claim that the analysis was fixed in advance unverifiable by anyone.

What this version contains is complete as it stands: the method and every choice its description does not determine (Section III), the baseline implementations and their notes (Section IV), one measured result in Table I, and the protocol (Section V) with its comparisons, statistics and correction fixed.

The next version adds the benchmark: Tables II and III populated, the cross-model transfer matrix, the budget sweep, and the RQ2 verdict under the test specified in Section V, whichever way it falls. The three design questions of Section III-F are settled there by the ablation rather than by us. Nothing in the protocol will be changed to accommodate what the numbers turn out to be; if something has to change, the change and its reason will be stated.

VIII. Conclusion

ADAPT-UAP combines four ideas that the data-free UAP literature developed separately: a ratio objective over the rectified and discarded halves of each activation, a spectral weighting that decides which layers that objective serves, synthetic crafting inputs carrying structure at several spatial scales at once, and a budget schedule that raises the constraint in stages and carries the perturbation's direction across each transition by rescaling. We have specified the method completely, including the decisions its description alone does not determine, implemented its three contested design choices as selectable modes so that the ablations settle them rather than the authors, and pre-registered the evaluation that will measure it. The implementation and the evaluation harness are released in full. What remains is to run it, and this paper will be revised when it has been.

Availability

Source, decision log, frozen preprocessing harness and the complete test suite are released with this version. The decision log records every non-obvious choice with its reason and its consequence if reversed, including the ones described in Section III-F and every divergence from a reference implementation in Section IV. The script that produces Table I is included and reproduces it on any machine, with no GPU, no dataset and no trained weights.

References

[1] S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi and P. Frossard, "Universal adversarial perturbations," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017.

[2] K. R. Mopuri, U. Garg and R. V. Babu, "Fast Feature Fool: a data independent approach to universal adversarial perturbations," in *Proc. British Machine Vision Conf. (BMVC)*, 2017.

[3] K. R. Mopuri, A. Ganeshan and R. V. Babu, "Generalizable data-free objective for crafting universal adversarial perturbations," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 10, pp. 2452–2465, 2019.

[4] Y. Liu et al., "TRM-UAP: enhancing the transferability of data-free universal adversarial perturbation via truncated ratio maximization," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023.

[5] Authors of PSP-UAP, "Pseudo-semantic prior regularization for data-free universal adversarial perturbation," 2024. (Full citation to be completed from the published record.)

[6] C. Zhang, P. Benz, A. Karjauv and I. S. Kweon, "Data-free universal adversarial perturbation and black-box attack," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021.

[7] Y. Yan and Y. Tian, "Data-free universal attack by exploiting the intrinsic vulnerability of deep models," in *Proc. AAAI Conf. Artificial Intelligence*, 2025.